

Testing

Trentin Tommaso

December 26, 2025

Contents

1	Idea	2
1.1	Verifica e validazione	2
1.2	Testing e debugging	2
2	Errore / Difetto / Malfunzionamento	2
2.1	Errore (Error)	2
2.2	Difetto (Fault)	2
2.3	Malfunzionamento (Failure)	2
2.4	Recap	2
3	Casi di test	3
3.1	Test suite	3
3.2	Oracolo	3
4	Obiettivi	3
4.1	Testing ideale / esaustivo	4
4.2	Test selection	4
4.3	Tesi di Dijkstra	4
4.4	Criteri terminazione / adeguatezza testing	4
5	Testing / Ispezione	4
5.1	Analisi statica	4
6	Livelli del testing	5
6.1	Test d'unità	5
6.1.1	Test d'unità automatizzato	5
6.2	Test d'integrazione	5
6.2.1	Test di regressione	5
6.2.2	Stub / Driver	5
6.3	Test di sistema	5
6.4	Test di accettazione	6

1 Idea

Testing = insieme di attività col fine di dimostrare che un programma svolge i compiti per il quale è stato realizzato, rilevando eventuali difetti prima di renderlo operativo.

1.1 Verifica e validazione

- **Verifica (Verification):** controllare che SW soddisfi la specifica dei requisiti / gli standard stabiliti all'inizio del ciclo di sviluppo;
- **Validazione (Validation):** dimostrare che SW risolva il giusto problema, supportando utilizzo atteso e soddisfacendo bisogni degli utenti nell'ambiente operativo.

1.2 Testing e debugging

Testing rientra nel processo più ampio di Verifica e Validazione (V&V), è un processo di esecuzione SW allo scopo di scoprire eventuali malfunzionamenti che indicherebbero difetti nel SW.

2 Errore / Difetto / Malfunzionamento

2.1 Errore (Error)

Situazione frutto di incomprensione da parte di singolo sviluppatore o team di sviluppo, esistono varie tipologie di errori che possono essere commessi:

- Nel tentativo di comprendere requisiti del problema;
- Nell'elaborare soluzione al problema;
- Nel comprendere / analizzare strumenti;
- Nella programmazione (dovuti a distrazione, superficialità...).

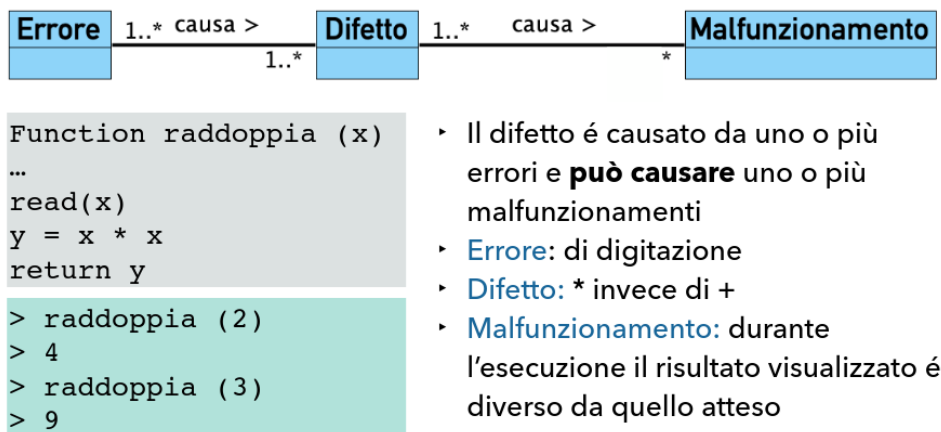
2.2 Difetto (Fault)

Manifestazione nel SW di un errore umano, ha natura *statica* (consiste in codice sintatticamente / semanticamente sbagliato). Se il codice difettoso non viene mai esercitato potrebbe passare inosservato in quanto non causerebbe alcun malfunzionamento durante l'esecuzione.

2.3 Malfunzionamento (Failure)

Incapacità del SW di comportarsi secondo aspettative / specifiche, ha natura *dinamica* in quanto accade solo in un certo istante e può essere osservato solo mediante esecuzione (più estremo è il crash, altri possono essere meno evidenti).

2.4 Recap



3 Casi di test

Durante il testing il SW è esercitato da un insieme di input e ne viene valutato il comportamento, un **test-case** è particolarmente utile se è in grado di scoprire un malfunzionamento non ancora scoperto da altri.

ID	Descrizione	Precondizioni	Valori di Input	Output atteso	Output Effettivo	Postcondizioni attese	Postcondizioni effettive	Esito
----	-------------	---------------	-----------------	---------------	------------------	-----------------------	--------------------------	-------

- **ID** aiuta a specificare / comunicare il caso di test;
- **Descrizione** può indicare requisito che il caso di test intende testare;
- **Precondizioni** sono condizioni da verificare per esecuzione test case;
- **Valori di input** indicano il modo in cui il programma sarà testato;
- **Valori di output attesi** rappresentano comportamento atteso dal SW dopo essere stato esercitato da input specificati;
- **Postcondizioni attese** rappresentano condizioni in cui il sistema dovrebbe trovarsi dopo esecuzione test case;
- **Output effettivo** e **Postcondizioni effettive** sono valori che vengono conosciuti solo dopo esecuzione del test case;
- **Esito** sarà *malfunzionamento rilevato* se 1+ valori di output / una postcondizione effettivi sono diversi da quelli attesi, *malfunzionamento non rilevato* altrimenti.

3.1 Test suite

Un caso di test TC_i rileva presenza di 1+ malfunzionamenti se programma P non è corretto per TC_i (produce risultati / postcondizioni diversi dagli attesi). Un insieme di test case detto **test suite** mira a rilevare presenza di più difetti possibili selezionando appropriatamente i test case.

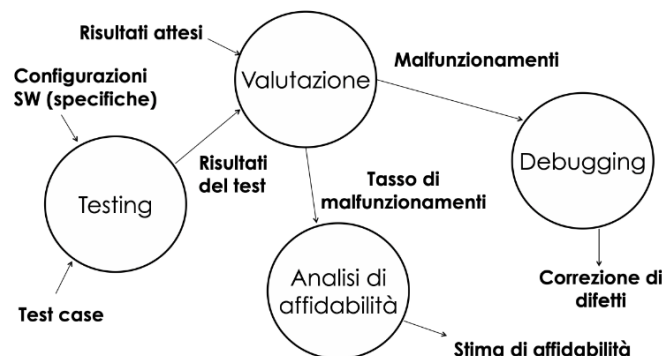
3.2 Oracolo

Conosce comportamento atteso per ogni test case e permette di confrontare risultato atteso con quello effettivo, può essere umano o automatico:

- Generato da specifiche formali;
- SW con stesse funzionalità ma sviluppato da altri;
- Versione precedente del SW (*test di regressione*).

4 Obiettivi

Testing rivela presenza di malfunzionamenti, non identifica i difetti. *Debugging* è processo di scoperta dei difetti a partire da malfunzionamenti rilevati, si occupa di identificare e rimuovere cause di un malfunzionamento. *Analisi dell'affidabilità* si occupa invece di fornire stima della probabilità che SW operi senza malfunzionamenti per un certo tempo in un certo ambiente.



4.1 Testing ideale / esaustivo

Il testing è **ideale** se assenza di malfunzionamenti rilevati implica correttezza del programma, è poi **esaustivo** se contiene tutte le possibili combinazioni dei dati in ingresso al programma (*un test esaustivo è ideale*). In casi non banali il test esaustivo non è pratico e risulta infattibile a causa dei costi elevati, in questi casi selezionano dei casi (**test selection**) che approssimino al meglio un *test ideale*.

4.2 Test selection

La bontà di una *test suite* si può misurare in termini di

- **Efficacia:** # malfunzionamenti trovati / # malfunzionamenti da trovare (quasi mai calcolabile in quanto numero da trovare sconosciuto, da privilegiare se si vuole SW affidabile);
- **Efficienza:** # test che rilevano malfunzionamenti / # test totali (da privilegiare se si vuole testing meno costoso).

L'ideale sarebbe trovare maggior numero possibile di malfunzionamenti con minor numero possibile di test case.

4.3 Tesi di Dijkstra

"Test possono dimostrare soltanto la presenza di errori, non la loro assenza". Testing pertanto non può dimostrare correttezza SW (in quanto problema indecidibile), deve fornire fiducia nel SW mostrando che è pronto a uso operativo. Un testing che determina assenza di malfunzionamenti può significare al contempo:

- Alta qualità del SW;
- Inadeguatezza dei casi di test.

4.4 Criteri terminazione / adeguatezza testing

Dato che testing esaustivo non fattibile vanno stabiliti criteri per decidere quando testing vada terminato e ritenuto adeguato.

- **Criterio temporale:** periodo di tempo predefinito;
- **Criterio di costo:** sforzo allocato predefinito;
- **Criterio di copertura:** numero predefinito di obiettivi (linee codice, scenari...) coperti;
- **Criterio statistico:** ultimi k test cases non hanno rilevato malfunzionamenti.

5 Testing / Ispezione

- *Testing* è un'analisi **dinamica** che mira a rilevare malfunzionamenti di un SW mediante osservazione del suo comportamento e confronto con quello atteso, è *possibile solo per SW che possa essere eseguito*.
- *Ispezione* è un'analisi **statica** che mira a verificare correttezza del SW senza eseguirlo (lettura codice), è possibile anche per SW incompleto o algoritmi non ancora implementati.

5.1 Analisi statica

- **Vantaggi**
 - Malfunzionamenti possono mascherarne altri durante esecuzione dinamica, con analisi statica si possono trovare subito;
 - Possono venire ispezionate versioni incomplete di un sistema senza costi aggiuntivi;
 - Ispezione può anche valutare attributi di qualità più generali come conformità a standard e portabilità.
- **Svantaggi**
 - Analisi statica può verificare conformità con specifiche ma non quella con necessità utenti;
 - Analisi statica meno adatta a verificare proprietà funzionali come usabilità.
 - Difficile rilevare malfunzionamenti dipendenti da valore assunto dinamicamente da variabili.

6 Livelli del testing

- **Test dei componenti (d'unità):** test di singole unità del SW;
- **Test d'integrazione:** integra diversi componenti tra loro per rilevare problemi d'interazione;
- **Test di sistema:** SW testato dopo integrazione di tutti i componenti;
- **Test di accettazione:** condotti da utente finale per validare i requisiti.

6.1 Test d'unità

Ciascuna unità viene verificata, spesso in fase di implementazione, in base alla propria documentazione. Le unità da testare possono essere:

- Metodi di una classe;
- Classi;
- Componenti riusabili che offrono specifica interfaccia per accedere alle loro funzionalità.

6.1.1 Test d'unità automatizzato

Test delle unità può essere oneroso, quindi hanno successo strategie di automatizzazione, la più banale è il **main-based approach** che fornisce a ogni unità un test method con codice per esercitare le funzionalità. Causa però **problemi**:

- main aggiuntivo sarà distribuito nel prodotto finale;
- per testare componenti alternativi saranno necessari più metodi nel main.

```
package calcolatrice;

public class Calcolatrice {

    public int somma(int a, int b) {
        return a+b;
    }
}

package calcolatrice;

public class Main1 {

    public static void main(String[] args) {
        Calcolatrice c=new Calcolatrice();
        int somma=c.somma(3, 39);
        int atteso=42;
        if (somma==atteso)
            System.out.println("OK");
        else
            System.out.println("Failure");
    }
}
```

6.2 Test d'integrazione

Richiede costruzione incrementale del sistema partendo da componenti testati singolarmente (aggiungendo una funzionalità alla volta): anche se quest'ultimi sono ritenuti corretti va verificata eventuale presenza di malfunzionamenti tra i moduli (possibili problemi delle interfacce).

6.2.1 Test di regressione

Possibile che dopo una modifica SW sia "regredito" (modifica abbia introdotto difetti prima non presenti), dopo ciascuna integrazione vanno quindi riefettuati i test precedenti per effettuare non ci siano nuovi malfunzionamenti del vecchio codice.

6.2.2 Stub / Driver

Moduli guida (**driver**) simulano i componenti chiamati, accettando dati del test case, invocando componente chiamato e stampando i risultati. I moduli fittizi (**stub**) invece simulano i componenti gerarchicamente inferiori al componente da testare: sono invocati dal componente testato, simulano funzionamento del componente chiamato mediante funzioni fittizie (correttezza vera per ipotesi). Driver e stub sono un overhead in quando devono essere implementati pur non facendo parte del prodotto finale.

6.3 Test di sistema

SW è testato dopo essere stato integrato (anche con altri elementi di sistema come HW e dati). Ha come scopo la verifica che tutti gli elementi del sistema siano correttamente integrati e che il sistema complessivo soddisfi i requisiti.

STUB E DRIVER: ESEMPIO

- Vogliamo testare la funzione `prod_scalare(v1, v2)` senza coinvolgere gli altri componenti che la utilizzano
- `prod_scalare(v1, v2)` invoca la funzione `prodotto(a,b)` ma non abbiamo ancora realizzato quest'ultima pur conoscendone la specifica
- Nel metodo **driver** scriviamo il codice per eseguire alcuni casi di test:

```
main(){  
    p = prod_scalare([2,4],[4,7]);  
    return p;  
}
```

- Sostituiamo a `prodotto` il suo metodo **stub**

```
prodotto(a,b) {  
    if (a==2 && b==4) return 8;  
    if (a==4 && b==7) return 28;  
}
```

6.4 Test di accettazione

Test condotti dall'utente finale per consentire validazione di tutti i requisiti:

- **α -Test** condotto da team selezionato di clienti e/o utilizzatori finali che avviene in ambiente controllato presso lo sviluppatore;
- **β -Test** condotto da numero maggiore di utenti finali con proprio HW che possono avere interazioni non previste con SW, sviluppatore solitamente non presente; oltre a essere una forma di commercializzazione può rivelare nuovi difetti / incompatibilità.